Maze Solving

Team Members : Mani Botla, Anissa Sollman and Nick Liverrett

Summary :

The maze problem involves finding a path from a starting position to a destination within a maze. In our program, mazes are represented by graphs which we traverse with DFS, BFS, A, and Ant Colony Optimization.*

Our output displays runtime and best path length for each algorithm. ACO has significantly more output, such as number of successful ants and count of unique paths, because there are multiple factors we must analyze to determine convergence of the ant colony.

With DFS, BFS, and A, we can compare these algorithms against each other by runtime and best path length. Whereas, with ACO, the algorithm takes significantly longer because we are testing how fast the ant colony learns to follow the same path. Therefore, we do not look at runtime for ACO as the primary performance metric, but rather we focus on convergence* behavior.

Overview:

The maze problem involves finding a path from a starting position to a destination within a maze. A maze will include obstacles or walls that prevent direct movement to the destination. A common analogy is a rat navigating through a maze to reach a block of cheese. It is usually represented using a 2D array or matrix. Each cell in the grid represents a location in the maze.

Cells may either be open or obstructed in our grid. If a cell of our grid is obstructed, it can be represented, for example, with a ‘1,’ and an unobstructed path can be represented with a ‘0.’ These characters, ‘0’ and ‘1,’ can be any character the programmer chooses so long as the logic remains.

There are many ways to look at the maze problem. Typically, we want to find the shortest path from our starting point to our destination, but other algorithms can analyze our maze in other manners. The four algorithms we use in this project are:

- Breadth First Search (BFS) : It is one of the standard algorithms for finding the shortest path in a maze making it a good baseline for comparison on other algorithms
- Depth First Search (DFS) : It helps illustrate how our search strategy can affect performance and results. DFS focuses on deep exploration, so it can sometimes find a solution faster, but does not guarantee optimality.
- A* : A* extends our shortest-path search to weighted mazes which is used in GPS routing with distances.
- Ant Colony Optimization(ACO) : ACO demonstrates how the maze problem can be solved based upon nature using ant swarm intelligence via pheromone trails, providing a comparison with our traditional deterministic algorithms.

Objective/Research Questions:

The goal of this experiment is to evaluate how different maze-solving algorithms perform under varying conditions, with a focus on understanding trade-offs between runtime, memory usage, and solution quality. One key research question is how the runtime of

Breadth-First Search (BFS), Depth-First Search (DFS), A* and Ant Colony Optimization (ACO) grows as the size of the maze increases. Since, ACO is a nature inspired algorithm which can be complex to implement, another key question was how does the ACO algorithm work. Another important question is which algorithm provides the best balance between execution time and the optimality of the path found. For the nature-inspired approach, the study will also explore how the solution quality of ACO changes as the number of iterations and ants increases, and whether improved accuracy comes at the cost of higher computational time. Finally, how does the ACO algorithm compare in terms of time efficiency for maze solving.

Description of experiment(s):

We are testing the algorithms DFS, BFS, A*, and ACO against each other using C++ for the algorithms and Python for the generated datasets.

The input size specifications for datasets:

Imperfect Small: 10 rows by 10 columns = 100 cells/nodes

Perfect Small: 31x31 = 961 cells/nodes

Imperfect and Perfect Medium: 250x250 = 62,500 cells/nodes

Periods represent open nodes that ants can take, and octothorpes represent walls which block ants.

The number of repeats we did were:

- 4 for DFS, BFS, and A*
- 2 for ACO since it only worked with the small size graphs

Regarding the factors that influenced ACO algorithm, each ant chooses its next move using a combination of pheromone strength, heuristic distance to the goal, and random exploration. Each cell is given a heuristic value based on Manhattan distance so that cells closer to the goal receive higher scores.

The Pheromone Factors were:

- Double alpha = 1.0 controls how strongly ants follow existing pheromone trails
- Double beta = 2.0 controls how strongly ants are influenced by the heuristic, or how close a cell is to the goal
- Double evaporation = 0.4 controls how quickly pheromone trails fade after each iteration (prevents early bad paths from stating too influential forever)
- Double exploreRate = 0.05 controls how often ants ignore the weighted pheromone/heuristic decision and choose a random valid move
- Int maxSteps = 4 * (width + height) limits how long each ant can search before stopping (prevents ants from wandering indefinitely through the maze)

Overall, the ACO algorithm balances exploitation and exploration by using pheromone trails to reinforce successful paths, a heuristic to guide ants toward the goal, evaporation to weaken poor or outdated trails, and limited random movement to prevent the colony from becoming trapped too early.

Another limitation of our project is the volume of output from the ACO algorithm. Since it takes an ant colony several iterations of the algorithm to build a stronger pheromone trail, we have to record several outcomes from each iteration, such as successful ant count, number of unique paths, and number of ants taking each unique path. This is a lot of data to represent via a graph, and it can take a considerable amount of ACO output/data to convey convergence.

The Characteristics of computer used to compile and execute program are:

- ASUS Gaming TUF F17

- ASUS TUF Gaming F17 - Tech Specs｜Laptops For Gaming｜ASUS USA

Operating System:

- Windows 10 Home

Processor:

- Intel® Core™ i5-10300H Processor 2.5 GHZ (8M Cache, up to 4.5 GHz, 4 cores)

Memory

- 8GB DDR4 SO-DIMM(2933MHz for i5-10300H/i7-10750H/i7-10870H), - Max Capacity:32GB

The primary performance metrics used in this experiment were execution time and final path length. Execution time was measured in microseconds and was used to evaluate the computational efficiency of each algorithm. Path length was used to evaluate the quality of the solution produced by each algorithm.

For DFS, BFS, and A*, direct comparisons could be made using runtime and path length because these algorithms are deterministic graph traversal algorithms designed to find a path in a single execution. ACO required additional evaluation metrics because it operates differently from the other algorithms. Unlike DFS, BFS, and A*. ACO is a probabilistic swarm-intelligence optimization algorithm that relies on multiple ants and repeated iterations to gradually converge toward strong solutions through pheromone reinforcement.

As a result, ACO generally required significantly greater execution time and a larger number of basic operations than the other algorithms. Therefore, execution time alone was not considered a sufficient measure of ACO performance.

Results:

Table 1: Results from using imperfect_small dataset

Algorithm	Found Path	Path Length	Runtime (in microseconds)	Optimal (Yes/No)
DFS	Yes	16	12	Yes
BFS	Yes	16	12	Yes
A*	Yes	16	26	Yes
Ant colony optimization (ACO)	Yes	16	67227	No

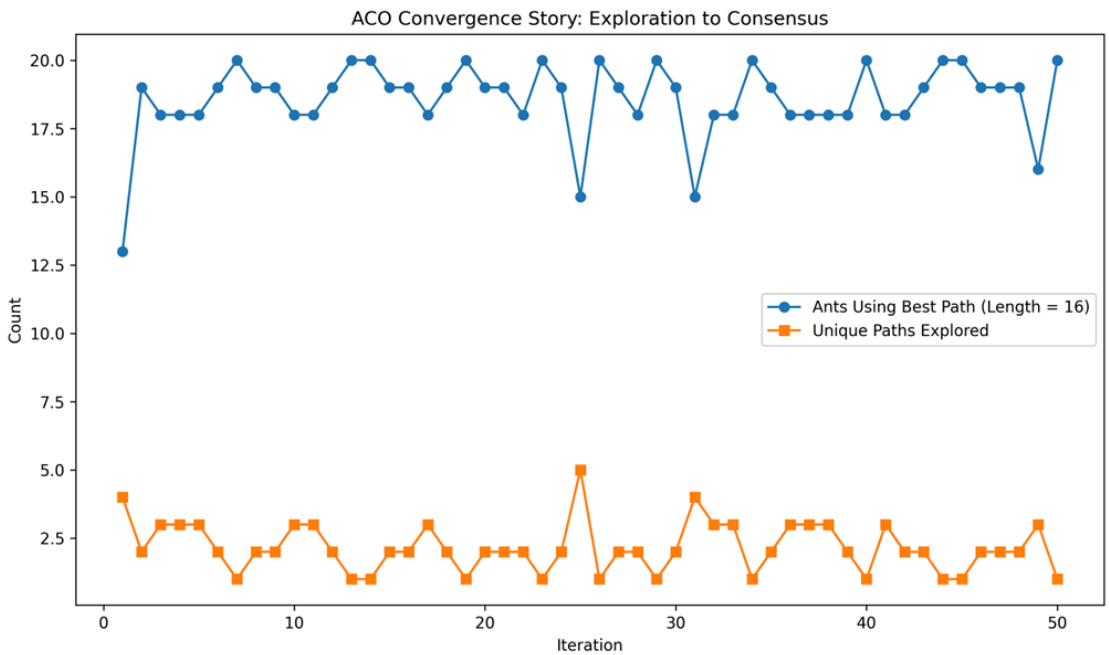

Figure 1: ACO convergence from using imperfect_small dataset

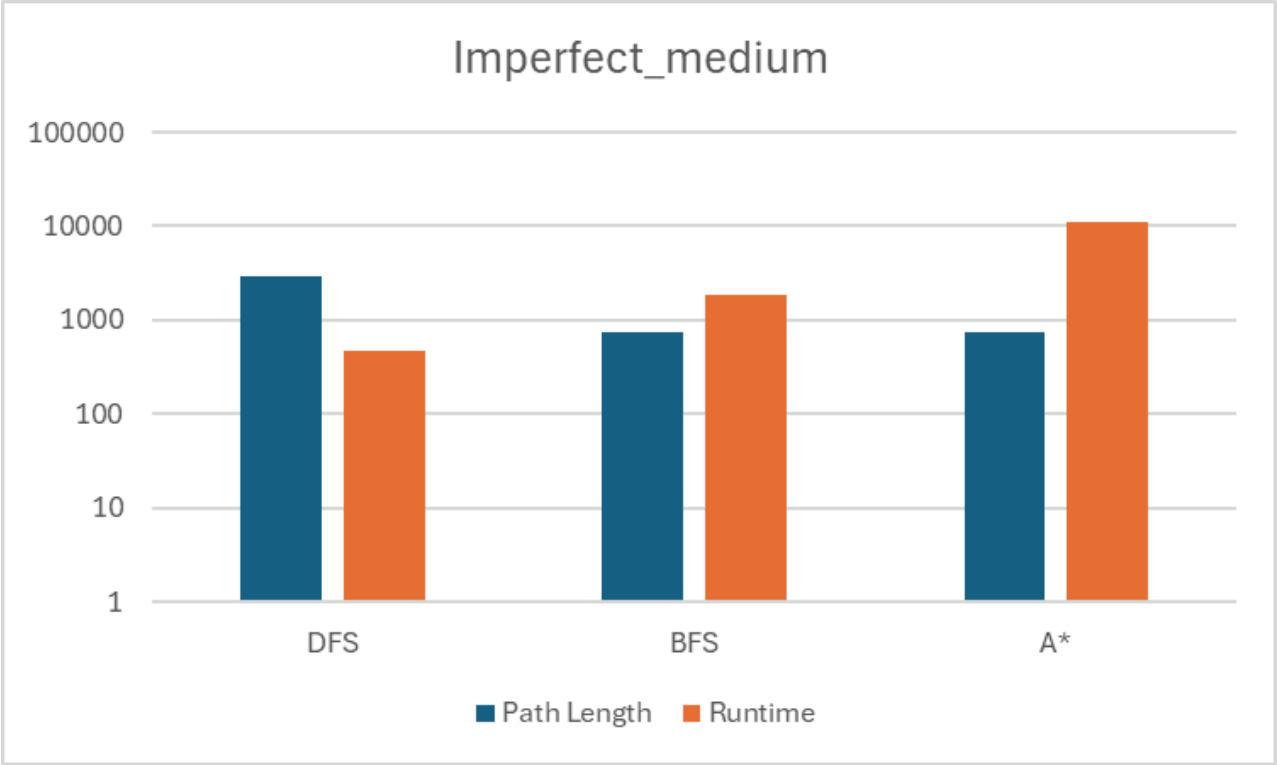

Figure 2: Runtime and Path length for imperfect_medium maze

Analysis :

According to table 1, all four algorithms successfully navigated the maze to find a path, though their efficiency and classifications varied significantly. DFS, BFS, and A* each achieved the optimal path length of 16, with DFA and BFS demonstrating the highest computational efficiency at a runtime of only 12 microseconds. A* followed closely behind with a runtime of 26 microseconds. While the Ant Colony Optimization (ACO) algorithm also reached a path length of 16, it required a disproportionately higher runtime of 67,227 microseconds. Despite finding the same path length as the others, ACO is categorized as non-optimal in this context, highlighting the significant computational overhead and stochastic nature of the swarm intelligence approach compared to the direct efficiency of traditional search algorithms on a small dataset.

According to figure 1, the Ant Colony Optimization (ACO) algorithm exhibited rapid and stable convergence when an imperfect_small maze dataset was used. During the early iterations, the ants explored a variety of possible routes, but by iterations 7–14, the colony had already begun to strongly favor the optimal path of length 16. After this point, the algorithm showed full convergence, with later iterations showing no meaningful deviation from the established best path. Even when the number of iterations was increased beyond 50 up to 75 or 100, the results remained unchanged. The best path length stayed fixed at 16, the number of successful ants remained consistently high, and the diversity of explored paths collapsed to a single dominant route. This shows that the colony had stabilized, and further iterations did not change or improve the solution.

According to figure 2, the search algorithms demonstrate a distinct trade-off between efficiency and optimal-path-finding when navigating the imperfect_medium dataset. Depth-First Search (DFS) proved to be the most time-efficient approach, boasting the lowest runtime. However, this speed came at the cost of solution quality, as it produced a path length higher than its counterparts. Conversely, both Breadth-First Search (BFS) and A* succeeded in finding a much more optimal path, with both algorithms settling on a length of approximately 750. Despite achieving the same path quality, their computational demands differed by an order of magnitude. While BFS required a runtime of 2,000 units, A* sat its runtime surge past 10,000 units. This indicates that in this specific environment, the heuristic used by A* likely struggled with the "imperfect" nature of the map, resulting in significantly higher overhead compared to the more straightforward BFS. ACO was not implemented perfectly for imperfect_medium and was not included in this comparison.

While the experiment provides a foundational look at algorithm behavior, several limitations hinder a definitive answer to the core research questions. The ACO algorithm was not implemented for medium datasets. This leaves a somewhat significant gap in the research, as we cannot observe how the nature-inspired approach scales its runtime and solution quality as the search space grows compared to traditional search methods. The empirical results for A* in Figure 2 indicate a potential mismatch between the heuristic used and the maze environment. An A* runtime that is five times slower than BFS suggests the heuristic was not "informed" enough, leading to high computational costs without providing the expected pruning of the search space.

In the future, develop some processes to ensure all algorithms, specifically ACO, are run across all datasets to produce a complete comparative picture of time efficiency and growth. Re-evaluate the A* heuristic to determine if different distance metrics make a more consistent heuristics that restores the efficiency of A* over BFS in medium environments. Perform more trials and build up larger results from a larger pool of datasets.

Next steps could be, At what specific maze size does the overhead of ACO become more efficient than traditional search (if ever)? How would the introduction of dynamic obstacles (moving walls) affect the convergence stability of ACO compared to the rigid paths found by BFS?

Sources:

Ant Colony Optimization - Intuition, Code & Visualization | Towards Data Science

Introduction to Ant Colony Optimization - GeeksforGeeks

Depth First Search or DFS for a Graph - GeeksforGeeks

[Breadth First Search or BFS for a Graph - GeeksforGeeks]

[A* Search Algorithm - GeeksforGeeks]

[The A* Algorithm: A Complete Guide | DataCamp]

Used AI for ACO convergence table to assist in what other columns can be added for better analysis

Used AI for the majority of Ant Colony Optimization algorithm since it is very complex and dense.

Used AI for A* algorithm as well.

Used AI for analysis of tables/algorithms.

Used AI for dataset generation script.

Team Member Contribution:

Team Member	Brainstorming	Coding	Data Collection	Presentation	Report Writing
Anissa Sollman	33.33	50	0	25	33.33
Mani Botla	33.33	15	0	75	33.33
Nick Liverett	33.33	35	100	0	33.33

Roadblocks:

We wanted to use multiple graph sizes to compare the algorithms, but the ACO uses pheromone factors which influence ant movement affecting the output for medium size graphs. If

we wanted ACO to work for small graphs, ants were not able to explore a larger size graph efficiently. If we did the opposite and improved the algorithm for medium-size graphs, ACO was too deterministic for small graphs. Ultimately, even if our ACO worked for the medium-size graphs, we would need a significant amount of data to show convergence.

Since it takes an ant colony several iterations of the algorithm to build a stronger pheromone trail, we have to record several outcomes from each iteration, such as successful ant count, number of unique paths, and number of ants taking each unique path.